Talent Matcher

A DSA-Driven Résumé–Job Compatibility & Recruitment Marketplace

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

in

A.Y. 2026-2027

Submitted by
Svara Haasini (2520090170)
B. Sudharshini (2520090227)

Team Number:

Section: 10

Under the guidance of

Ms.K.Chandusha

Asst.prof, CS&IT



Hyderabad-500090, Telangana, India.

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



DECLARATION

The Mini Project Report entitled "**Talent Matcher: A DSA-Driven Résumé–Job Compatibility & Recruitment Marketplace**" is a record of Bonafide work of **Svara Haasini – 2520090170, B. Sudharshini – 2520090227** submitted in partial fulfillment for the award of B.Tech in Computer Science and Engineering to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

Svara Haasini – 2520090170

B. Sudharshini – 2520090227

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



CERTIFICATE

This is to certify that the mini project based report entitled "**Talent Matcher: A DSA-Driven Résumé–Job Compatibility & Recruitment Marketplace**" is a bonafide work done and submitted by **Svara Haasini – 2520090170, B. Sudharshini – 2520090227** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year **2026-2027.**

Signature of the Supervisor

Signature of the HOD **Signature of the External Examiner**

ACKNOWLEDGEMENT

The success of this project would not have been possible without the timely help and guidance of many people. We wish to express our sincere thanks to everyone who assisted us, in one way or another, in completing this project.

Our greatest appreciation goes to our guide, <<Guide Name>>, <<Designation>>, <<Department>>, whose tremendous support, encouragement and guidance for this project cannot be expressed in words.

We express our gratitude to **Dr. P. Venkateshwar Rao**, Head of the Department of Computer Science Engineering, for providing us with adequate facilities, ways and means by which we were able to complete this project-based lab.

We thank all the members of the teaching and non-teaching staff, and everyone who assisted us directly or indirectly in the successful completion of this project. Finally, we sincerely thank our parents, friends and classmates for their kind help and cooperation during our work.

Svara Haasini – 2520090170
B. Sudharshini – 2520090227

TABLE OF CONTENTS

S.No.	Contents	Page no.
1.	Abstract	6
2.	Introduction	7
3.	Literature survey	8
4.	Problem Statement and Objectives	10
5.	Hardware and Software requirements	11
6.	Implementation	12
7.	Results & Discussion	15
8.	Conclusion & Future Scope	18
9.	References	19

1.ABSTRACT

Talent Matcher is a web-based recruitment platform that applies classical data structures and algorithms to the practical problem of matching candidates to jobs. Students and recruiters sign up for separate, role-restricted accounts. Recruiters post jobs, bulk-upload résumés (PDF, DOCX, TXT, CSV or ZIP) and review candidates ranked by a transparent weighted compatibility score, while students browse postings, maintain a profile and track their applications. Behind the screens, the Hungarian algorithm and an Edmonds–Karp max-flow network solve optimal candidate–job assignment; Greedy Set Cover and a bitmask dynamic program form the smallest team that covers the required skills; Knuth–Morris–Pratt and Aho–Corasick search résumé text for skills; and Wagner–Fischer edit distance suggests corrections for misspelled skills. Spring Boot, Spring Security and Spring Data JPA provide real authentication and an H2 database, so accounts, candidates and jobs persist across restarts. Every ranking can be traced to an explicit calculation, which makes the system explainable rather than a black box. The project shows how course concepts such as hashing, graphs, dynamic programming and string matching solve a real hiring workflow from end to end.

Team Name: Talent Matcher

Team Logo (if any): Not applicable

Team Members:

1. Svara Haasini, 2520090170

2. B. Sudharshini, 2520090227

2. INTRODUCTION

Hiring at any real scale means comparing many résumés against many open roles, and doing so consistently. Screening a stack of résumés by hand is slow, differs from one reviewer to the next, and makes it hard to justify why one candidate was shortlisted over another. Students, on the other side, get very little insight into why a job is or is not a good fit for them.

Talent Matcher addresses both sides of this problem in one application. Its core is not a black-box model but a set of well-understood, analyzable algorithms and data structures, which are the material covered in this course. Candidates and jobs are modelled as structured objects; skills are normalized and indexed for fast lookup; compatibility is a transparent weighted formula; assignment and team formation are solved with combinatorial optimization algorithms; and résumé text is searched with string-matching algorithms rather than guesswork.

On top of this algorithmic core, the project is built as a genuine multi-user web application. Students and recruiters have separate accounts with hashed passwords, a real database keeps accounts, candidates and jobs across restarts, and the server refuses to show a student the recruiter pages and vice versa. The interface is deliberately simple: a student sees three tabs (Browse jobs, My applications, My profile) and a recruiter sees two (Recruiter workspace, Upload résumés), with the remaining tools placed inside those pages.

The rest of this report is organized as follows. Section 3 surveys similar platforms, Section 4 states the problem and objectives, Section 5 lists the hardware and software requirements, Section 6 presents four implementation modules, Section 7 shows the running application, Section 8 concludes with future scope, and Section 9 lists the references.

3. LITERATURE SURVEY

The following existing platforms solve a similar candidate–job matching problem and informed the design of this project.

1. LinkedIn Talent Solutions (LinkedIn Recruiter). LinkedIn Recruiter lets recruiters search a very large professional network using skills, titles, experience and location filters, and it recommends people who look like a good fit for an open role. Its strength is a structured skill taxonomy that makes profiles comparable across companies. Talent Matcher adopts the same idea at a much smaller scale through its skill normalizer and alias dictionary, so that "JS" and "JavaScript" count as one skill. The difference is that our ranking is an open, explainable formula instead of a closed model.

2. Naukri (Resdex résumé database). Naukri's recruiter tools are built around Resdex, a searchable résumé database in which recruiters filter candidates by keywords, experience, location and qualification. It shows that keyword and filter search over résumé text is the foundation of any bulk-screening feature. We build that foundation ourselves, using Knuth–Morris–Pratt for single-keyword search and Aho–Corasick to find every dictionary skill in a résumé in one pass, rather than depending on an external search service.

3. Indeed (Hiring and résumé search). Indeed lets employers search and filter résumés and see candidates who match a posted job. Showing the match against a job's stated requirements, instead of presenting a bare ranking, helps recruiters trust the result. Talent Matcher follows the same principle: for every candidate it displays the matched skills, the missing skills and a written explanation of how the weighted compatibility score was calculated.

4. Greenhouse (applicant tracking system). Greenhouse organizes hiring into defined stages with structured interview scorecards, so every candidate is judged against the same criteria. Our application workflow adopts the same staged pipeline: an application moves through the statuses Applied, Shortlisted, Interview, Selected and Rejected, and only the recruiter who owns a posting can change them.

5. Workday Recruiting. Workday is an enterprise HR suite in which recruiters, hiring managers and candidates each see a different, permission-controlled view of the same hiring data. This motivated the strict recruiter–student separation in Talent Matcher: the two account types have different menus, and the server rejects any attempt by one role to open the other's pages instead of merely hiding the links.

6. HackerRank for Work. HackerRank screens candidates with automated technical assessments and gives recruiters a ranked list before human interviews begin. It illustrates the value of combining an automated score with a human decision. Talent Matcher likewise produces an automatic compatibility ranking, but leaves shortlisting, interviewing and the final selection to the recruiter.

7. Zoho Recruit. Zoho Recruit is an applicant tracking and recruitment management system that parses uploaded résumés into candidate records and lets recruiters match them to open jobs. Its upload, extract, review and save workflow is similar to our bulk résumé import, in which each uploaded file becomes an editable draft profile that the recruiter reviews before it is added to the candidate pool.

Across these systems four ideas recur and were adopted in this project: the match should be explainable; recruiter and candidate views must be strictly separated; fast text matching over résumé content is essential for bulk screening; and parsed data should be reviewed by a person before it is saved. What sets Talent Matcher apart is that its matching, assignment, team-formation and search features are built directly on the classical algorithms studied in this course. The URLs of the referred sites are listed in Section 9.

4. Problem Statement and Objectives

Problem Statement: Manually screening many résumés against a job is slow and inconsistent, and it gives recruiters no transparent way to justify why one candidate ranks above another.

Problem Description: A recruiter with dozens of résumés for a single opening needs a fast, repeatable way to shortlist the best candidates, while a student needs to know which skills they are missing for the roles they want.

Objectives:

Identify the Project's Purpose: The project was started to replace slow, subjective résumé screening with a transparent, algorithm-based matching system, wrapped in a real multi-user web application. Its objectives are:

- Model candidates, jobs and applications as structured data and index skills for fast lookup.
- Score every candidate–job pair with a transparent, weighted and explainable compatibility formula.
- Solve one-to-one and capacity-constrained assignment optimally (Hungarian algorithm, Edmonds–Karp max-flow).
- Form the smallest candidate team that covers a set of required skills (Greedy Set Cover, bitmask DP).
- Search résumé text efficiently and tolerate misspelled skill names (KMP, Aho–Corasick, Wagner–Fischer).
- Provide real, separate, persisted Student and Recruiter accounts with server-side access control.
- Let a recruiter bulk-upload many résumés at once and see the candidates ranked against a chosen job.

5. HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements:

- Processor: dual-core 2 GHz or higher (any modern laptop or desktop)
- RAM: 4 GB minimum, 8 GB recommended
- Storage: about 500 MB free (project, dependencies and the local H2 database file)
- Display and network: a screen of 1366 × 768 or larger and an internet connection for the first build

Software Requirements:

- Operating system: Windows 10/11, Linux or macOS
- Java Development Kit (JDK) 17 or higher and Apache Maven 3.9+
- Spring Boot 3.5.16 (Web MVC, Thymeleaf, Validation, Security, Data JPA)
- H2 database (embedded, file-based; no separate database server needed)
- Apache PDFBox 3.0.8 and Apache POI 5.5.1 for PDF and DOCX résumé text extraction
- Thymeleaf templates with Bootstrap 5.3.8 and custom CSS (bundled locally; no Node build step)
- JUnit 5 and MockMvc for automated testing
- A modern web browser (Chrome, Edge or Firefox)

6. IMPLEMENTATION

Four representative modules of the project are presented below: the compatibility scoring engine, the Aho–Corasick skill search, the authentication and role-separation layer, and the résumé extraction used by the bulk import.

Module 1 – Compatibility Scorer (core matching engine)

Computes a weighted 0–100 score for a candidate–job pair from the share of matched skills, the experience ratio and the qualification level, and returns the matched and missing skills with a readable explanation.

```
public MatchResult score(Candidate c, Job j) {
  HashSet<String> required = j.requiredSkills(), possessed = c.skills();
  Set<String> matched = new HashSet<>(), missing = new HashSet<>();
  for (String skill : required) {
    if (possessed.contains(skill)) matched.add(skill);
    else missing.add(skill);
  }
  double s = required.isEmpty() ? 100 : 100.0 * matched.size() / required.size();
  double e = j.minimumExperience() == 0 ? 100
      : Math.min(100, 100 * c.yearsOfExperience() / j.minimumExperience());
  double q = c.qualification().ordinal() >= j.requiredQualification().ordinal()
      ? 100 : 0;
  // default weights: skills 0.7, experience 0.2, qualification 0.1
  double total = skillsWeight * s + experienceWeight * e + qualificationWeight * q;
  String explanation = ...;   // human-readable breakdown of the score
  return new MatchResult(c, j, total, s, e, q, matched, missing, explanation);
}
```

Module 2 – Aho–Corasick skill search

Builds a trie of every known skill with BFS failure links, so all skills in a résumé are found in a single left-to-right pass, including overlapping names.

```
public AhoCorasick(Collection<String> patterns) {
  nodes.add(new Node());
  for (String raw : new TreeSet<>(patterns)) {          // 1. build the trie
    String p = raw.toLowerCase(Locale.ROOT);
    int state = 0;
    for (char ch : p.toCharArray()) { /* add child node if missing */ }
    nodes.get(state).output.add(p);
  }
  Queue<Integer> queue = new ArrayDeque<>(nodes.get(0).next.values());
  while (!queue.isEmpty()) {                            // 2. BFS failure links
    int parent = queue.remove();
    for (var edge : nodes.get(parent).next.entrySet()) {
      char ch = edge.getKey();
      int child = edge.getValue(), f = nodes.get(parent).failure;
      while (f != 0 && !nodes.get(f).next.containsKey(ch)) f = nodes.get(f).failure;
      nodes.get(child).failure = nodes.get(f).next.getOrDefault(ch, 0);
      nodes.get(child).output.addAll(nodes.get(nodes.get(child).failure).output);
      queue.add(child);
    }
  }
}

public List<Hit> search(String input) {                 // 3. one pass over the text
  String text = input.toLowerCase(Locale.ROOT);
  List<Hit> hits = new ArrayList<>();
  int state = 0;
  for (int i = 0; i < text.length(); i++) {
    char ch = text.charAt(i);
    while (state != 0 && !nodes.get(state).next.containsKey(ch))
      state = nodes.get(state).failure;
    state = nodes.get(state).next.getOrDefault(ch, 0);
    for (String p : nodes.get(state).output)
      hits.add(new Hit(p, i - p.length() + 1, i + 1));
  }
  return hits;
}
```

Module 3 – Authentication and recruiter / student separation

Signs users in from the database with BCrypt-hashed passwords and rejects, on the server, any request by one role for the other role's pages.

```
// AppUserDetailsService: sign-in against the database (passwords are BCrypt hashes)
public UserDetails loadUserByUsername(String username) {
  UserAccount u = users.findByUsername(username)
      .orElseThrow(() -> new UsernameNotFoundException("Unknown user: " + username));
  return User.withUsername(u.getUsername())
      .password(u.getPassword())
      .authorities("ROLE_" + u.getRole().name())     // RECRUITER or CANDIDATE
      .disabled(!u.isEnabled())
      .build();
}

// RoleAccessInterceptor: keeps the recruiter and student modules separate
if (recruiterArea && role != Role.RECRUITER) {
  response.sendError(HttpServletResponse.SC_FORBIDDEN,
      "This section is for recruiter accounts.");
  return false;
}
if (studentArea && role != Role.CANDIDATE) {
  response.sendError(HttpServletResponse.SC_FORBIDDEN,
      "This section is for student accounts.");
  return false;
}
```

Module 4 – Résumé extraction for bulk import

Reads PDF, DOCX, TXT and CSV files (ZIP archives are unpacked first), enforces size limits and returns an editable draft per résumé, or a friendly failure message for unreadable files.

```
public List<ResumeDraft> extract(ResumeInput input) {
  try {
    if (input.bytes().length == 0) throw new IllegalArgumentException("Empty file");
    if (input.bytes().length > MAX_FILE)
      throw new IllegalArgumentException("File exceeds 5 MB");
    String name = input.name().toLowerCase(Locale.ROOT), text;
    if (name.endsWith(".pdf")) {                       // PDF -> PDFBox
      try (var doc = Loader.loadPDF(input.bytes())) {
        if (doc.getNumberOfPages() > 50)
          throw new IllegalArgumentException("PDF exceeds 50 pages");
        text = new PDFTextStripper().getText(doc);
      }
    } else if (name.endsWith(".docx")) {               // DOCX -> Apache POI
      checkDocx(input.bytes());
      try (var doc = new XWPFDocument(new ByteArrayInputStream(input.bytes()));
          var extractor = new XWPFWordExtractor(doc)) {
        text = extractor.getText();
      }
    } else if (name.endsWith(".txt") || name.endsWith(".csv")) {
      text = decodeUtf8(input.bytes());   // strict UTF-8, strips the BOM
    } else throw new IllegalArgumentException("Use PDF, DOCX, TXT or CSV.");
    if (name.endsWith(".csv")) return csv(input.name(), text);  // one draft per row
    return List.of(parse(input.name(), text));   // skills found with Aho-Corasick
  } catch (Exception e) {
    return List.of(ResumeDraft.failure(input.name(), friendlyMessage(e)));
  }
}
```

7. RESULTS & DISCUSSION

The application was run locally at http://localhost:8080 and tested through its student and recruiter flows. The main screens are shown below, each with a short explanation.

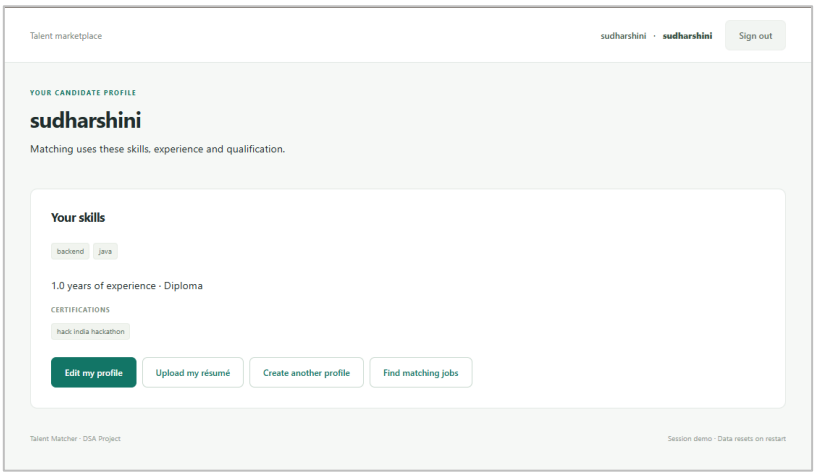


Fig 7.3 – My profile (student). The signed-in student's skills, experience, qualification and certifications, which are the data used for all matching.

Testing the application with the bundled sample résumés gave consistent results. The Hungarian algorithm produced the optimal one-to-one assignment, and Greedy Set Cover returned teams that matched the exact bitmask result on small inputs. The Aho–Corasick scan found every dictionary skill in a résumé in a single pass, including overlapping names such as "java" and "javascript". Signing in as a student and opening a recruiter address (or the reverse) was rejected by the server with an access-denied response.

8. CONCLUSION & FUTURE SCOPE

Conclusion

The project set out to replace manual, subjective résumé screening with a transparent, algorithm-based system inside a genuine web application, and each objective was met. Candidates, jobs and applications are modelled as structured data with a skill index; every candidate–job pair receives a weighted, explainable compatibility score; the Hungarian algorithm and Edmonds–Karp solve assignment; Greedy Set Cover and a bitmask dynamic program form teams; KMP, Aho–Corasick and Wagner–Fischer handle résumé text search and spelling suggestions; and separate Student and Recruiter accounts are stored in a database and enforced on the server.

In short, the work shows that classical data structures and algorithms can drive a real, usable hiring workflow, and that the resulting rankings stay explainable to the people who rely on them.

Future Scope

- **Near term:** richer section-aware résumé parsing, e-mail notifications when an application status changes, and export of ranked shortlists to PDF or Excel.
- **Medium term:** store job-posting details and applications in the database as well (today accounts, candidates and jobs persist, while posting status and applications are kept in memory), and keep an audit log of recruiter decisions.
- **Long term:** a learning-to-rank layer on top of the existing transparent score, kept as an explainable aid rather than a replacement, and multi-tenant deployment with per-organization data isolation.

9. REFERENCES

1. LinkedIn Talent Solutions – https://business.linkedin.com/talent-solutions

2. Naukri.com (Resdex résumé database for recruiters) – https://www.naukri.com/

3. Indeed for Employers – https://www.indeed.com/hire

4. Greenhouse applicant tracking system – https://www.greenhouse.com/

5. Workday Recruiting – https://www.workday.com/

6. HackerRank for Work – https://www.hackerrank.com/

7. Zoho Recruit – https://www.zoho.com/recruit/

8. Spring Boot reference documentation – https://docs.spring.io/spring-boot/

9. Spring Security reference documentation – https://docs.spring.io/spring-security/reference/

10. Apache PDFBox – https://pdfbox.apache.org/

11. Apache POI – https://poi.apache.org/

12. H2 Database Engine – https://www.h2database.com/

13. T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, 4th ed., MIT Press, 2022.